

Rockchip AOV Development Guide

ID: RK-KF-YF-A17

Release Version: V1.1.0

Release Date: 2024-10-14

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS” . ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily provides the basic functions and development guide for AOV (Always On Video).

Product Version

Chipset	Kernel Version
RV1106	Linux 5.10
RV1106B	Linux 5.10

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineers

Software Development Engineers

Revision History

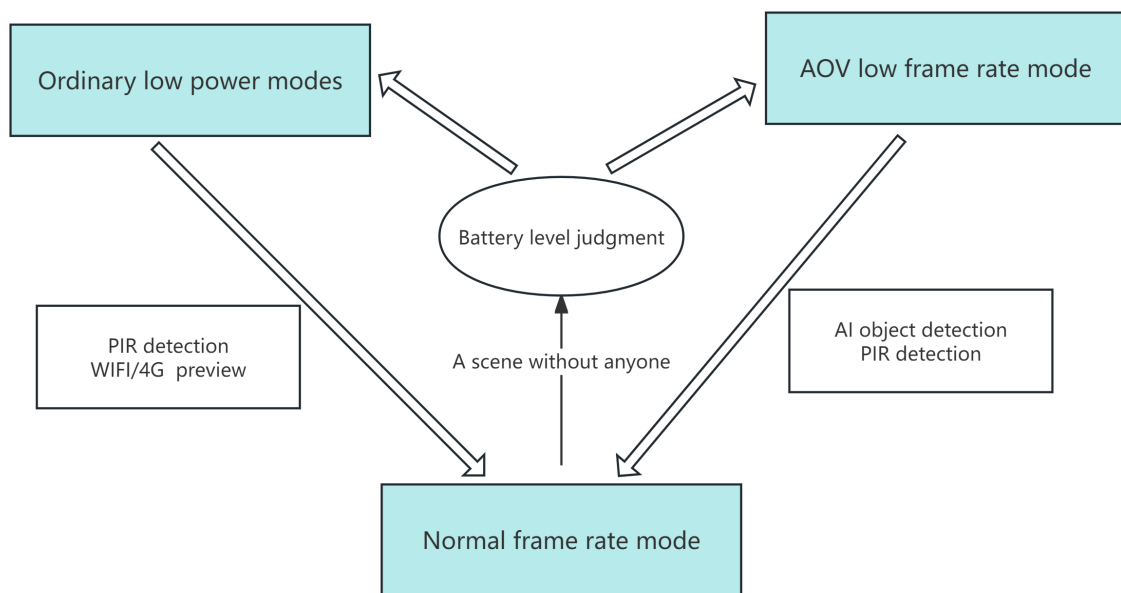
Version Number	Author	Modification Date	Modification Description
V1.0.0	YYZ, HL, HKH, CWW	2023-12-12	Initial version
V1.0.1	XY, MLC	2023-12-25	Added WiFi/4G, camera driver instructions
V1.0.2	ZY	2024-01-10	Added binocular instructions
V1.0.3	HKH	2024-02-19	Added sleep state clock source instructions
V1.0.4	HKH	2024-04-27	Updated debugging instructions
V1.1.0	ZY	2024-10-14	1. Added eMMC instructions 2. Added binocular top-bottom non-blending stitching demo instructions 3. Added WiFi/4G, camera driver instructions 4. Added power consumption data for RV1106B

Table of Contents

Rockchip AOV Development Guide

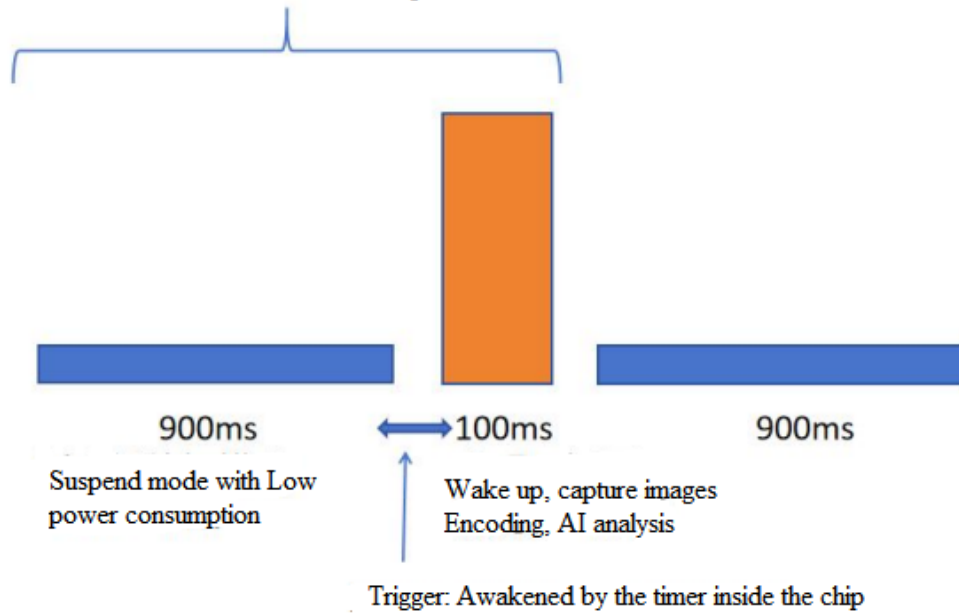
1. AOV Overview
 - 1.1 Scenario Introduction
 - 1.2 AOV Process Introduction
2. Hardware Introduction
 - 2.1 Hardware Interface Introduction
 - 2.1.1 RV1106 Hardware Interface
 - 2.1.2 RV1106B Hardware Interface
3. Development Guide
 - 3.1 AOV State Switching
 - 3.1.1 Set AOV Suspend Time
 - 3.1.2 Switch from Normal Mode to AOV Mode
 - 3.1.3 Switching from AOV Mode to Normal Mode
 - 3.1.4 AOV Mode
 - 3.2 AOV Configuration
 - 3.2.1 System Configuration
 - 3.2.1.1 Kernel
 - 3.2.1.2 BoardConfig
 - 3.2.2 Button Wakeup Configuration
 - 3.2.3 Camera Driver
 - 3.2.3.1 Standby Mode
 - 3.2.3.2 Power-Down Mode
 - 3.2.3.3 Fill Light Control
 - 3.2.4 Wi-Fi/4G Development Introduction
 - 3.2.5 Clock Source in Sleep Mode
 - 3.2.6 AOV Development Guide Based on eMMC
 - 3.2.6.1 Board-Level Configuration
 - 3.2.6.2 Application Process
4. Debugging Introduction
 - 4.1 Demo Introduction
 - 4.1.1 ISP Stream Capture Test
 - 4.1.2 Main Stream Encoding Test
 - 4.1.3 Human Detection Test
 - 4.1.4 AIISP Testing
 - 4.1.5 Dual Camera ISP Stream Capture Test
 - 4.1.6 Dual Camera Main Stream Encoding Test
 - 4.1.7 Dual-Camera Human Detection Test
 - 4.1.8 Dual Camera Vertical Non-Blend Stitching Test
 - 4.2 Sleep&Wake-up Latency Troubleshooting
 - 4.2.1 AOV Time Confirmation
 - 4.2.2 Software Latency Troubleshooting
 - 4.3 Streaming Issues Troubleshooting
5. Typical Power Consumption
 - 5.1 Power Consumption During Basic Sleep
 - 5.1.1 RV1106
 - 5.1.2 RV1106B
 - 5.2 RV1106 SC200AI data
 - 5.2.1 Encoding a Frame + Human Detection
 - 5.2.2 AIISP + Encoding a Frame + Human Detection
 - 5.3 RV1106B SC450AI Performance
 - 5.3.1 Encode one frame + human detection
 - 5.4 RV1106B Dual-SC200AI Performance
 - 5.4.1 Non-blending stitching + encode one frame + human detection
6. Frequently Asked Questions
 - 6.1 How to Add Sleep Time to Application Layer Timing

- 6.2 How to Obtain the Wakeup Source
- 6.3 Why the SD Card is Not Recognized After AOV SDK Boot
- 6.4 How to Resolve Data Transmission Issues During AOV Process with SD Card or 4G When Data is Temporarily Unavailable
- 6.5 How to Control Infrared Lights with Lower Power Consumption



- Normal Frame Rate Mode: Entered during remote preview, PIR event, and human detection event. Requires recording, networking, and other functions;
- AOV Low Frame Rate Mode: When idle, with sufficient battery, the SOC records the main stream at a low frame rate, and WiFi/4G stays active;
- Ordinary Low Power Mode: When idle, with insufficient battery, the SOC powers down while WiFi/4G remains active.

Detailed introduction to each frame's operation in AOV mode

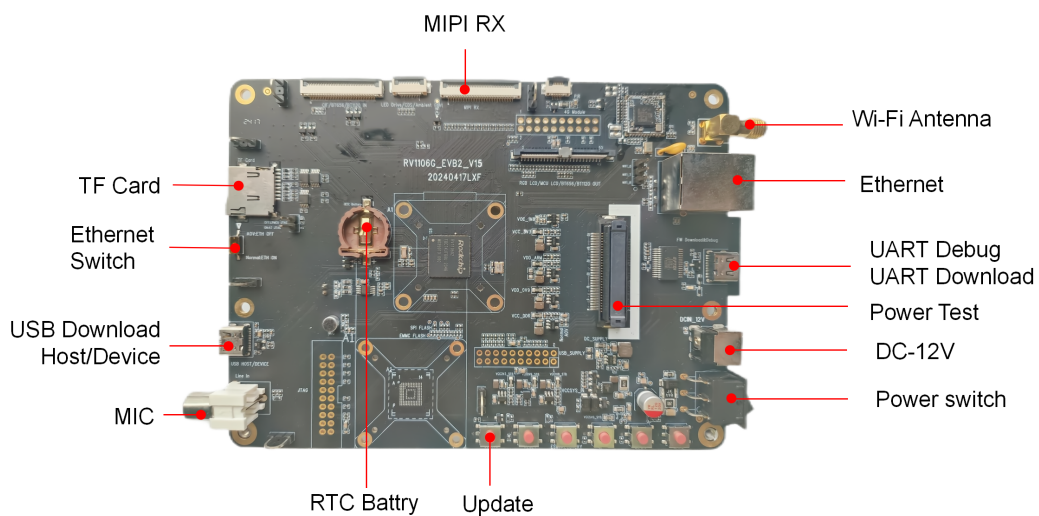


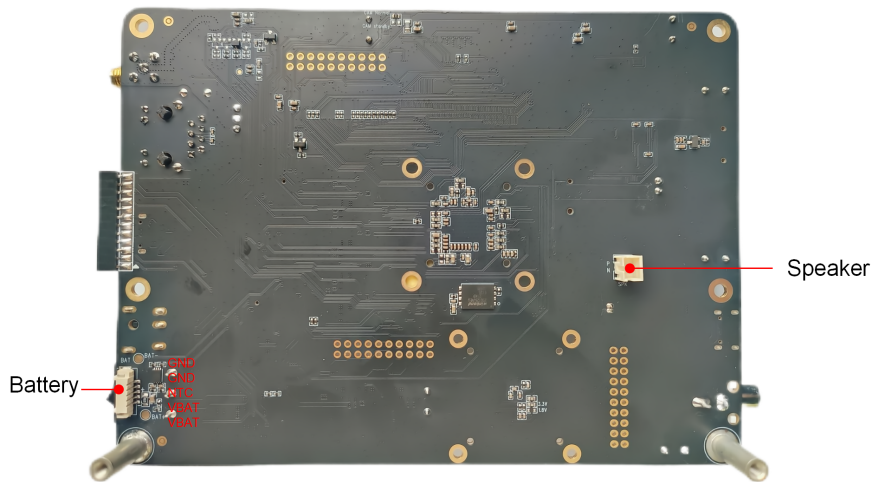
!Introduction-1](./resources/Introduction-1.png)

2. Hardware Introduction

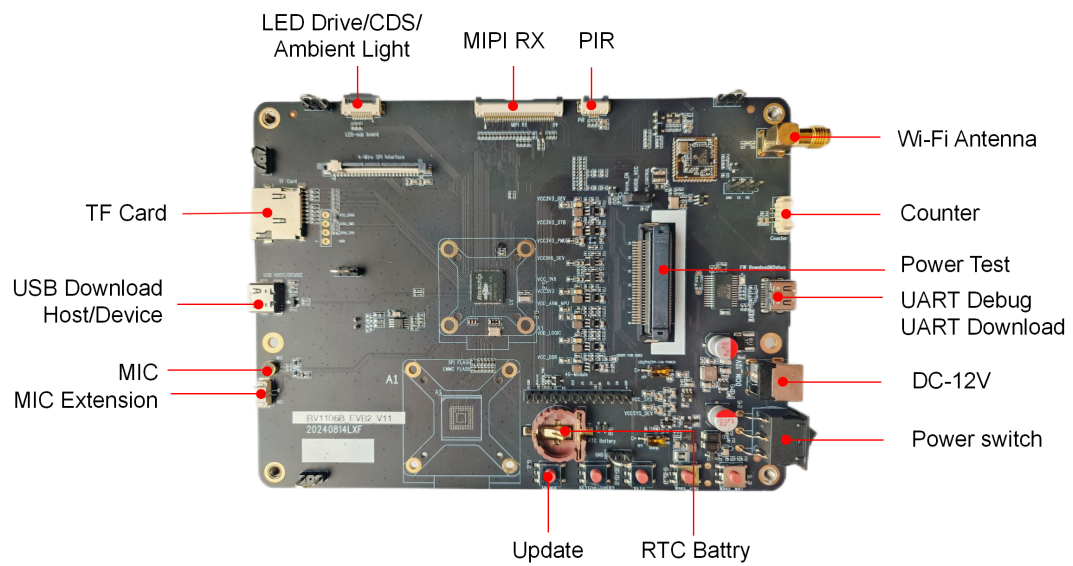
2.1 Hardware Interface Introduction

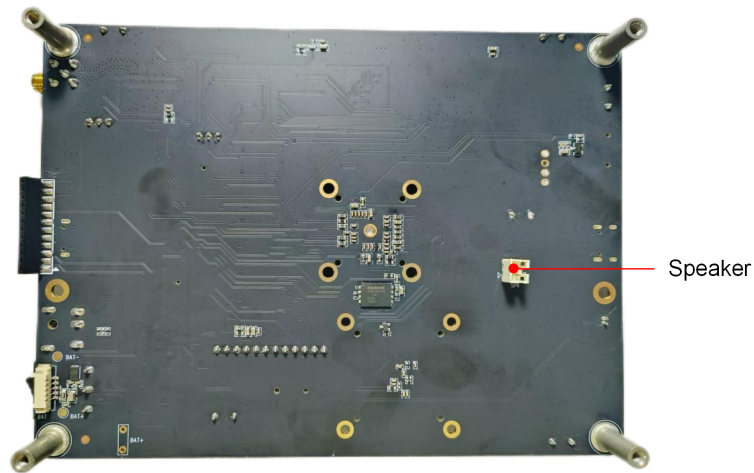
2.1.1 RV1106 Hardware Interface





2.1.2 RV1106B Hardware Interface





3. Development Guide

3.1 AOV State Switching

3.1.1 Set AOV Suspend Time

Supports up to 37 hours, unit ms, for details refer to section 3.2.5.

Choose one of the following interfaces. If you need a fixed sleep of 1s, use the second one. If you need a frame interval of 1s, use the second one.

```
// Set the sleep time
int SAMPLE_COMM_AOV_SetSuspendTime(int wakeup_suspend_time);
// Set the frame interval
int SAMPLE_COMM_AOV_SetPeriodTime(int wakeup_period_time);
```

3.1.2 Switch from Normal Mode to AOV Mode

Switching from the normal frame rate to AOV mode requires reducing power by disabling media data streams and external devices other than AOV recording, as follows:

1. To switch from the normal frame rate mode to AOV single-frame mode, it is necessary to fetch the remaining frames from the system to prevent residual frames from affecting the sleep&wake-up cycle.


```

SAMPLE_COMM_ISP_SingleFrame(0);
// drop frame
VIDEO_FRAME_INFO_S stViFrame_tmp;
while (RK_MPI_VI_GetChnFrame(ctx->s32DevId, ctx->s32ChnId, &stViFrame_tmp, 1000) ==
      RK_SUCCESS) {
    RK_MPI_VI_ReleaseChnFrame(ctx->s32DevId, ctx->s32ChnId, &stViFrame_tmp);
}

```

2. Disable media data streams such as sub-streams and audio that are not needed in AOV mode.

```

# Suspend sub-stream
RK_MPI_VI_PauseChn(ctx->vi[2].u32PipeId, ctx->vi[2].s32ChnId);
# Turn off audio
TBD

```

3. Unload peripherals to reduce power during sleep&wake-up.

SD Card:

```

int SAMPLE_COMM_AOV_UnbindSdcard(); // Unload SD Card

```

Ethernet:

```

int SAMPLE_COMM_AOV_UnbindEthernet(); // Unload Ethernet

```

Audio Driver:

```

int SAMPLE_COMM_AOV_UnbindSoundcard(); // Unload Audio

```

USB Driver:

```

int SAMPLE_COMM_AOV_UnbindUsb(); //Unload USB driver

```

SDIO Driver:

```

int SAMPLE_COMM_AOV_UnbindSDIO(); // Unload SDIO driver

```

4. Enter AOV mode.

```

void SAMPLE_COMM_AOV_EnterSleep();

```

3.1.3 Switching from AOV Mode to Normal Mode

The primary focus is on the resuming of media data streams and peripherals. For details, please refer to the following items:

1. Resuming Multi-Frame Mode

```

RK_S32 SAMPLE_COMM_ISP_MultiFrame(int cam_id);

```

2. Resuming Sub-Stream, Audio, and Other Media Data Streams

```
# Resume sub-stream
RK_MPI_VI_ResumeChn(ctx->vi[2].u32Pipeld, ctx->vi[2].s32ChnId);
# Initialize AI/AO
TBD
```

3. Resuming Peripherals

SD Card:

```
int SAMPLE_COMM_AOV_BindSdcard(); // Mount SD Card
```

Ethernet:

```
int SAMPLE_COMM_AOV_BindEthernet(); // Mount Ethernet
```

Audio Driver:

```
int SAMPLE_COMM_AOV_BindSoundcard(); // Mount Audio Driver
```

USB driver

```
int SAMPLE_COMM_AOV_BindUsb();
```

SDIO driver

```
int SAMPLE_COMM_AOV_BindSDIO(); // Mount SDIO driver
```

3.1.4 AOV Mode

The AOV mode allows for the storage of the bitstream until it accumulates to a certain value, after which the bitstream is written to an SD card or transmitted via WIFI at once. The following is a reference for writing to an SD card:

```
SAMPLE_COMM_AOV_CopyStreamToSdcard
```

After the bitstream is obtained or after human detection, immediately enter the AOV mode. If you want to achieve lower power consumption, you can first extract the YUV and then perform encoding + IVS + human detection after waking up from sleep:

```
SAMPLE_COMM_AOV_EnterSleep();
```

If the product is concerned with the time it takes for brightness to converge, the following process can be used to quickly converge brightness:

```
// 1. Check whether there is big changes
RK_BOOL SAMPLE_COMM_ISP_IsExpBigChange(int cam_id); // Returns TRUE if there is a significant
change in environmental brightness
// 2. Switch to continuous frame mode if there is a big change
// 3. Determine if convergence has occurred in continuous frame mode
RK_BOOL SAMPLE_COMM_ISP_IsExpConverge(int cam_id); // Returns TRUE after brightness has
converged
// 4. After convergence, switch to AOV mode
```

3.2 AOV Configuration

3.2.1 System Configuration

3.2.1.1 Kernel

1. Configuration

RV1106:

arch/arm/configs/rv1106-pm.config

arch/arm/configs/rv1106-wakeup.config

RV1106B:

arch/arm/configs/rv1106b-pm.config

arch/arm/configs/rv1106b-wakeup.config

2. PM-GPIO Configuration

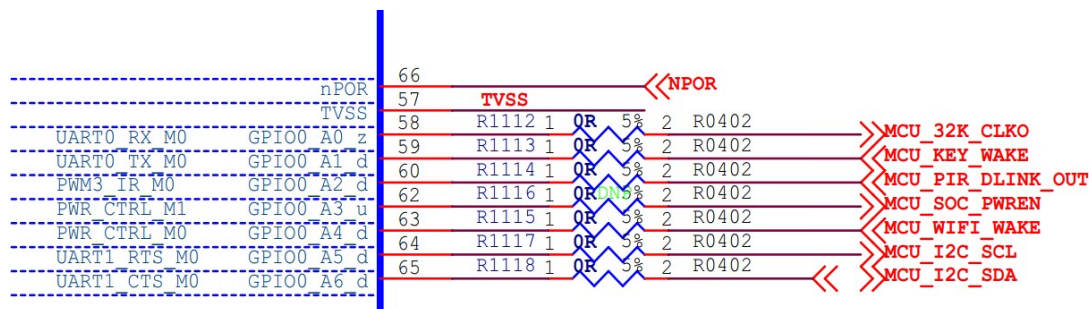
After entering sleep mode, only PMUIO remains powered, while all other regular I/Os are powered down. Therefore, it is necessary to configure GPIO0A0 to GPIO0A6 as pull-up or pull-down based on the state that the peripheral needs to maintain during standby. If the GPIO pin is not connected to a peripheral, it should be configured as pull-down. The following is a reference configuration for EVB2-V13:

```
rockchip_suspend: rockchip-suspend {
    compatible = "rockchip,pm-rv1106";
    status = "okay";
    rockchip,sleep-io-config = <
        (0
            | RKPM_IO_CFG_IOMUX_GPIO
            | RKPM_IO_CFG_GPIO_DIR_INPUT
            | RKPM_IO_CFG_PULL_DOWN
            | RKPM_IO_CFG_ID(0)
        )
        (0
            | RKPM_IO_CFG_IOMUX_GPIO
            | RKPM_IO_CFG_GPIO_DIR_INPUT
            | RKPM_IO_CFG_PULL_DOWN
            | RKPM_IO_CFG_ID(1)
        )
        (0
            | RKPM_IO_CFG_IOMUX_GPIO
```

```

| RKPM_IO_CFG_GPIO_DIR_INPUT
| RKPM_IO_CFG_PULL_DOWN
| RKPM_IO_CFG_ID(2)
)
(0
| RKPM_IO_CFG_IOMUX_GPIO
| RKPM_IO_CFG_GPIO_DIR_INPUT
| RKPM_IO_CFG_PULL_UP
| RKPM_IO_CFG_ID(3)
)
(0
| RKPM_IO_CFG_IOMUX_GPIO
| RKPM_IO_CFG_GPIO_DIR_INPUT
| RKPM_IO_CFG_PULL_UP
| RKPM_IO_CFG_ID(4)
)
(0
| RKPM_IO_CFG_IOMUX_GPIO
| RKPM_IO_CFG_GPIO_DIR_INPUT
| RKPM_IO_CFG_PULL_NONE
| RKPM_IO_CFG_ID(5)
)
(0
| RKPM_IO_CFG_IOMUX_GPIO
| RKPM_IO_CFG_GPIO_DIR_INPUT
| RKPM_IO_CFG_PULL_NONE
| RKPM_IO_CFG_ID(6)
)
>;
};

```



3. Regular I/O

If the activated peripheral is connected to a regular I/O, the I/O of these peripherals should be switched to high impedance to prevent leakage to the corresponding power domain, which could cause abnormalities.

3.2.1.2 BoardConfig

Add the following two configurations to BoardConfig.mk:

```

export RK_APP_TYPE=RK_SAMPLE_AOV
export RK_ENABLE_AOV=y

```

3.2.2 Button Wakeup Configuration

1. Open the `rv1106-pm.config` and `rv1106-wakeup.config` configurations in `BoardConfig.mk`, for example, in `project/cfg/BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106_EVB2_V11-BAT_IPC_AOV.mk`:

```
export RK_KERNEL_DEFCONFIG_FRAGMENT="rv1106-tb.config rv1106-battery-ipc.config rv1106-pm.config rv1106-wakeup.config rv1106-sdiowifi.config"
```

2. Configure the kernel

Enable GPIO button wakeup (corresponding to key2 on the EVB2). It is important to ensure that the initial state of this button is consistent with the configuration in PM-GPIO, otherwise it may cause a false wakeup.

```
diff --git a/arch/arm/boot/dts/rv1106g-evb2-v10.dts b/arch/arm/boot/dts/rv1106g-evb2-v10.dts
index 41e31555a203..f4e758927766 100644
--- a/arch/arm/boot/dts/rv1106g-evb2-v10.dts
+++ b/arch/arm/boot/dts/rv1106g-evb2-v10.dts
@@ -50,6 +50,29 @@ @ wireless_wlan: wireless-wlan {
        WIFI,host_wake_irq = <&gpio1 RK_PB0 GPIO_ACTIVE_HIGH>;
        status = "okay";
    };
+
+ gpio-keys {
+     compatible = "gpio-keys";
+     autorepeat;
+     pinctrl-names = "default";
+     pinctrl-0 = <&pwr_key>;
+     power-key {
+         gpios = <&gpio0 RK_PA1 GPIO_ACTIVE_HIGH>;
+         linux,code = <KEY_POWER>;
+         label = "GPIO Key Power";
+         debounce-interval = <100>;
+         wakeup-source;
+         /* gpio-key,wakeup; */
+     };
+ };
+};
+
+&pinctrl {
+     buttons {
+         pwr_key:pwr-key {
+             rockchip,pins = <0 RK_PA1 RK_FUNC_GPIO &pcfg_pull_down>;
+         };
+     };
+};
diff --git a/arch/arm/configs/rv1106-battery-ipc.config b/arch/arm/configs/rv1106-battery-ipc.config
index c9f8f88003a3..07c6f00f7c5a 100644
--- a/arch/arm/configs/rv1106-battery-ipc.config
+++ b/arch/arm/configs/rv1106-battery-ipc.config
@@ -268,3 +267,6 @@ CONFIG_SPI_ROCKCHIP_SFC=y
# CONFIG_VIDEO_S5C73M3 is not set
CONFIG_ZLIB_DEFLATE=y
CONFIG_ZLIB_INFLATE=y
+CONFIG_INPUT=y
```

```
+CONFIG_INPUT_KEYBOARD=y
+CONFIG_KEYBOARD_GPIO=y
```

If the trigger mode needs to be changed to a single edge, add the following to the `power-key` node in the dts:

```
wakeup-event-action = <EV_ACT_ASSERTED>; // Rising edge trigger
wakeup-event-action = <EV_ACT_DEASSERTED>; // Falling edge trigger
```

3.2.3 Camera Driver

AOV cameras support two modes: Standby and power-off. If the camera supports the low-power consumption of Standby, it is recommended to choose the Standby mode, which will simplify development. For development details, please refer to the document [Rockchip_Trouble_Shooting_Linux5.10_Camera_CN.pdf](#).

3.2.3.1 Standby Mode

Taking SC200AI as an example, when there is stream out, it enters the normal mode, and when the stream is stopped, it enters the low-power standby mode. Cameras have different characteristics, and it is best to communicate with the manufacturer to confirm clearly.

3.2.3.2 Power-Down Mode

Restores the camera state by software, and on the hardware side, the three voltages do not need to be supplied in the sleep state. However, since the camera state is restored by the software, the overall working duty cycle of AOV will be longer, and the duration is related to the I2C write register sequence.

3.2.3.3 Fill Light Control

The power consumption of the light is relatively high, and the camera is not always operating in AOV mode. To reduce the duty cycle of the light, in other word, to reduce the power consumption of the light, it is necessary for the driver to accurately control the switching of the light.

3.2.4 Wi-Fi/4G Development Introduction

Hardware Requirements:

- Wi-Fi/4G Modules: One is a Wi-Fi/4G_WAKE_GPIO with wake-up capable, the other is a standard Wi-Fi/4G_NORMAL_GPIO;
- RV1106: One is a RK_WAKE_GPIO with wake-up capable, the other is a standard RK_NORMAL_GPIO;
- It is recommended to use low-power Wi-Fi/4G modules that support an independent network protocol stack internally, can independently connect to external servers, maintain a connection in sleep mode, and support network wake-up, such as Wi-Fi: Hisilicon Hi3861L, 4G Module: Quectel Air780E;

Wake-up Methods:

- RK_NORMAL_GPIO is connected to WiFi/4G_WAKE_GPIO: The purpose is to allow RK controller to actively wake up the WiFi/4G module;
- WiFi/4G_NORMAL_GPIO is connected to RK_WAKE_GPIO: The purpose is to allow the WiFi/4G module to actively wake up RK controller;

Application Scenarios:

1. WiFi/4G is responsible for basic data communication and supports the controller to maintain a connection with an external server independently after entering sleep mode, in a low-power mode, and can be remotely woken up by the server or actively woken up by RK controller through a GPIO interrupt;
2. Before the controller enters sleep mode, the following operations are required:
 - First, send a sleep command (such as an AT command) to the WiFi/4G module to keep it in low-power mode;
 - If network wake-up is required, the remote server address/port must also be sent to the WiFi/4G to allow it to connect to the remote server;
 - Finally, the WiFi/4G driver should be unloaded with `rmmmod`, the purpose is to ensure that during sleep&wake-up, The kernel no longer needs to handle WiFi/4G driver and device enumeration issues;
3. When the controller enters a sleep cycle once every second, there are several scenarios at this time:
 - The controller is woken up by a PIR or other peripherals, but there is no need for WiFi/4G communication, so no action is required for WiFi/4G;
 - The controller is woken up by a PIR or other peripherals and needs to connect to the WiFi network for communication. At this point, the RK controller needs to wake up the WiFi module by **pulling up/down the RK_NORMAL_GPIO connected to the WiFi/4G module**, and then the controller needs to reload the KO of WiFi/4G and configure the IP address to go online; (Note: Since WiFi/4G is only in low-power mode, the network is always connected, so it can quickly connect to the network after being woken up)
 - The WiFi/4G is woken up by the remote server, and the WiFi/4G wakes up the RK controller by **pulling up/down the WIFI/4G_NORMAL_GPIO connected to the controller**, At this point, the main control needs to reload the WiFi/4G KO and configure the IP address to go online;
4. Repeat step 2;

WiFi/4G Software Process Reference

Obtain the detailed process and steps from the following address:

FTP Address: <ftp://www.rockchip.com.cn>

Note: FTP software must support Chinese encoding

Username: rkwifi

Password: Cng9280H8t

/11-Linux平台/WIFI低功耗方案/AOV

3.2.5 Clock Source in Sleep Mode

In sleep mode, the clock source can be selected as the PVTM divider or the RTC output clock. The PVTM scheme may have certain errors, which are influenced by factors such as temperature, voltage, and frequency, thereby affecting the sleep duration. The SDK is PVTM scheme by defaults.

RTC Scheme:

Hardware:

It is necessary to enable the RTC (only applicable to RV1106, RV1103 does not have a built-in RTC). For details, please refer to the hardware reference design.

Kernel Configuration:

```
&rtc {
    rockchip,rtc-suspend-bypass;
    status = "okay";
};

&rockchip_suspend {
    rockchip,sleep-mode-config = <
        (0
        | RKPM_SLP_32K_EXT
        )
    >;
    status = "okay";
};
```

```
CONFIG_RTC_DRV_ROCKCHIP=y
```

3.2.6 AOV Development Guide Based on eMMC

The AOV development process based on eMMC is similar to that of SPI NOR Flash. Due to the long recovery time of eMMC during sleep&wake-up processes (about 1-2 seconds), it significantly affects the AOV operating power consumption. Therefore, when developing AOV applications, eMMC should be used like an SD card. Unbind the device driver when entering single-frame mode and rebind the device driver when returning to multi-frame mode to reduce the time of eMMC driver sleep&wake-up. Additionally, since it is necessary to package the required kernel, applications, and dependent libraries into the ramfs during runtime, more memory is needed compared to SPI NOR. When packaging the file system, efforts should be made to minimize the size of the ramfs image.

3.2.6.1 Board-Level Configuration

Board-level configuration can refer to `project/cfg/BoardConfig_IPC/BoardConfig-EMMC-NONE-RV1106_EVB2_V12-IPC_AOV.mk`. It is necessary to package the kernel and application, library files into the boot partition and set it as initramfs (read-write) or romfs (read-only) to reside in memory. Then other files such as IQ files, AI model files, etc., can be packaged into the userdata partition.

```
export
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),256K(uboot),32M(boot),1G(userdata),-
(media)"
# boot can choose between initramfs or romfs, boot partition resides in memory
export RK_PARTITION_FS_TYPE_CFG=boot@IGNORE@initramfs,userdata@/userdata@ext4
# Comment out this configuration to package oem into the boot partition
# export RK_BUILD_APP_TO_OEM_PARTITION=y
```


The following compilation configuration is required to added to the kernel:

```
diff --git a/arch/arm/configs/rv1106-wakeup.config b/arch/arm/configs/rv1106-wakeup.config
index 605fe719b696..e060517a7492 100644
--- a/arch/arm/configs/rv1106-wakeup.config
+++ b/arch/arm/configs/rv1106-wakeup.config
@@ -1,3 +1,4 @@
+CONFIG_BLK_DEV_INITRD=y
# CONFIG_ETHERNET is not set
CONFIG_HIGH_RES_TIMERS=y
CONFIG_INPUT=y
```

3.2.6.2 Application Process

The application process can refer to sample_aov_vi.c. Before entering single-frame mode, it is necessary to first unmount the ext4 file system and then unbind the eMMC device driver. After the above series of operations, the ext4 file system will not be accessible, and the system will run on ramfs. When restoring to multi-frame mode, the corresponding recovery operations are performed.

```
static void enter_single_frame_mode(RK_S32 s32DevId) {
    RK_S32 ret = RK_SUCCESS;
    if (g_bEnableEmmc) {
        // 0. sync filesystem
        sync();
        // 1. umount filesystem
        ret = umount2("/userdata", MNT_DETACH);
        if (ret != RK_SUCCESS)
            RK_LOGE("umount /userdata failed %s", strerror(errno));
        // 2. unbind emmc device
        ret = SAMPLE_COMM_AOV_UnbindEmmc();
        if (ret != RK_SUCCESS)
            RK_LOGE("unbind emmc failed %s", strerror(errno));
    }
    // 3. enter single frame mode
    // When booting, the default is multi-frame mode, so you need to pause the
    // stream and pick up the remaining frames
    SAMPLE_COMM_ISP_SingleFrame(s32DevId);
    SAMPLE_COMM_AOV_DisableNonBootCPUs();
}
```

Before restoring to multi-frame mode, first bind the device driver and wait for the eMMC driver to complete the probe operation. This step is packaged in the SAMPLE_COMM_AOV_BindEmmc function. Then, it is necessary to mount the previously unmounted ext4 file system.

```
static void enter_multi_frame_mode(RK_S32 s32DevId) {
    RK_S32 ret = RK_SUCCESS;
    // 1. enter multi frame mode
    SAMPLE_COMM_ISP_MultiFrame(s32DevId);
    SAMPLE_COMM_AOV_EnableNonBootCPUs();
    if (g_bEnableEmmc) {
        // 2. bind emmc device
        ret = SAMPLE_COMM_AOV_BindEmmc();
        if (ret != RK_SUCCESS)
```

```

RK_LOGE("bind emmc failed %s", strerror(errno));
// 3. mount filesystem
ret = mount("/dev/mmcblk0p5", "/userdata", "ext4", 0, NULL);
if (ret != RK_SUCCESS)
    RK_LOGE("mount /userdata failed %s", strerror(errno));
}
}

```

In the `mount` function, the partition name passed in is related to the `RK_PARTITION_CMD_IN_ENV` parameter in BoardConfig. The compilation script will automatically generate partition information based on this parameter. For example, in the reference configuration BoardConfig-EMMC-NONE-RV1106_EVB2_V12-IPC_AOV.mk, the userdata partition is the fifth, corresponding to the device name `/dev/mmcblk0p5`, and so on.

```

export
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),256K(uboot),32M(boot),1G(userdata),-
(media)"

```

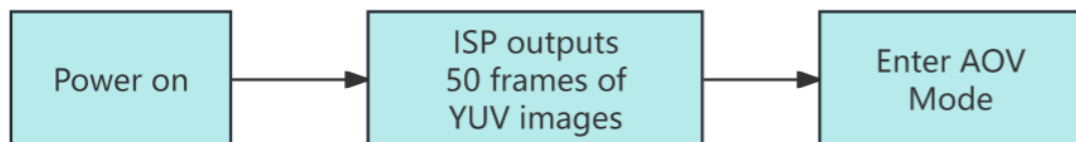
4. Debugging Introduction

4.1 Demo Introduction

4.1.1 ISP Stream Capture Test

Description:

Capture a frame from the ISP and then enter sleep mode.



Code Path:

project/app/aov_sample/sample_aov_vi.c

Quick Usage:

```

sample_aov_vi -w 1920 -h 1080 -a /etc/iqfiles/ --aov_loop_count 50 --boot_frame 50 --
suspend_time 1000

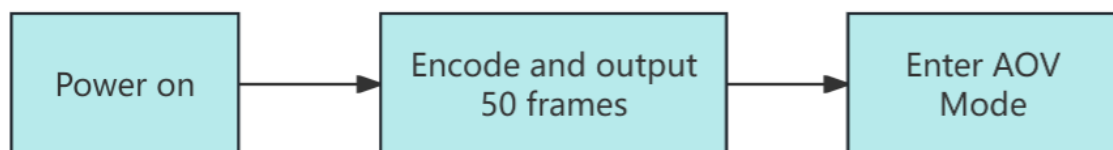
```

Option	Description	Default
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP functionality. If this option is used, the built-in ISP functionality is enabled. If no parameter is provided, the default value is used, and the parameter is the path to the AIQ file folder.	NULL
--suspend_time	System sleep duration in milliseconds	1000
--aov_loop_count	Set the number of AOV frames to output before exiting the program	-1
--boot_frame	Set the number of frames to run after the program starts before entering AOV mode	60
--output_path	The output path for YUV, with the file name vi_0.bin	NULL
--vi_frame_mode	Single frame or multi-frame mode switch. 0: Single frame mode, 1: Single frame -> Multi-frame -> Single frame	0

4.1.2 Main Stream Encoding Test

Description

Take a frame from the ISP and send it for encoding. After the encoding is complete, the system enters a sleep state. If an SD card is inserted, the encoded stream will be stored in /mnt/sdcard/wakeup_frame_%d, with a threshold of 10M. If there are more than 100 folders, manual deletion is required.



Code Path:

project/app/aov_sample/sample_aov_vi_venc.c

Quick Usage:

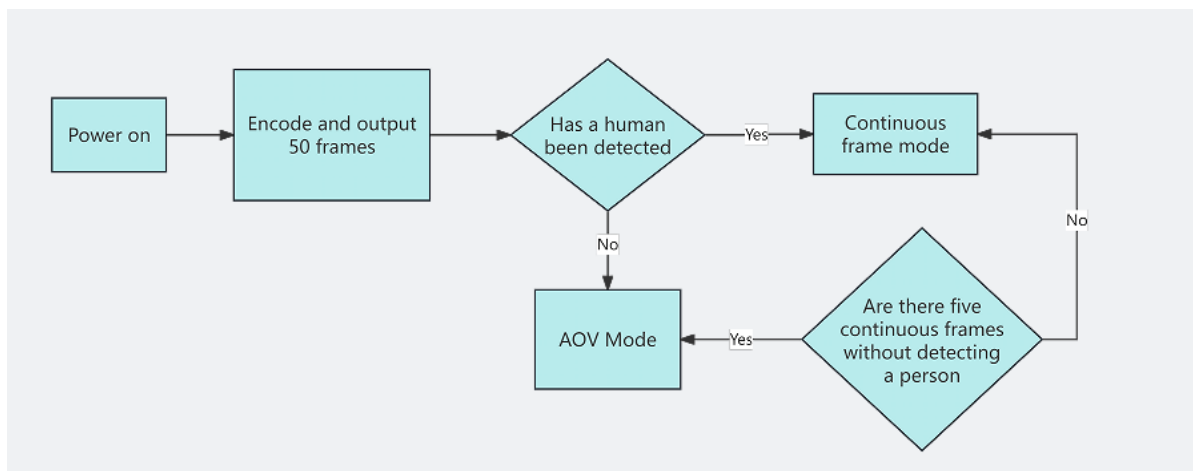
sample_aov_vi_venc -w 1920 -h 1080 -a /etc/iqfiles/

Option	Description	Default Value
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP function. If this option is used, the built-in ISP function is enabled. If no parameter is provided, the default value is used, and the parameter is the path to the AIQ file folder.	NULL
-e --encode	Set the encoding method	h264cbr
-b --bitrate	Set the encoding bitrate	4096
-r --wrap	Wrap switch, 0: off, 1: on	0
--wrap_lines	Wrap line setting, 0: height/2, 1: height/4, 2: height/8	1
--suspend_time	System sleep duration in milliseconds	1000
--aov_loop_count	Set the number of AOV output frames before exiting the program	-1
--boot_frame	Set the number of frames after which the program enters AOV mode after running	60

4.1.3 Human Detection Test

Description:

Take a frame from the ISP and send it to the NN for human recognition. If a human is detected, switch to continuous frame mode. Wait until 5 consecutive frames do not detect a human before entering sleep mode.



Code Path:

project/app/aov_sample/sample_aov_vi_iva_venc.c

Quick Usage:

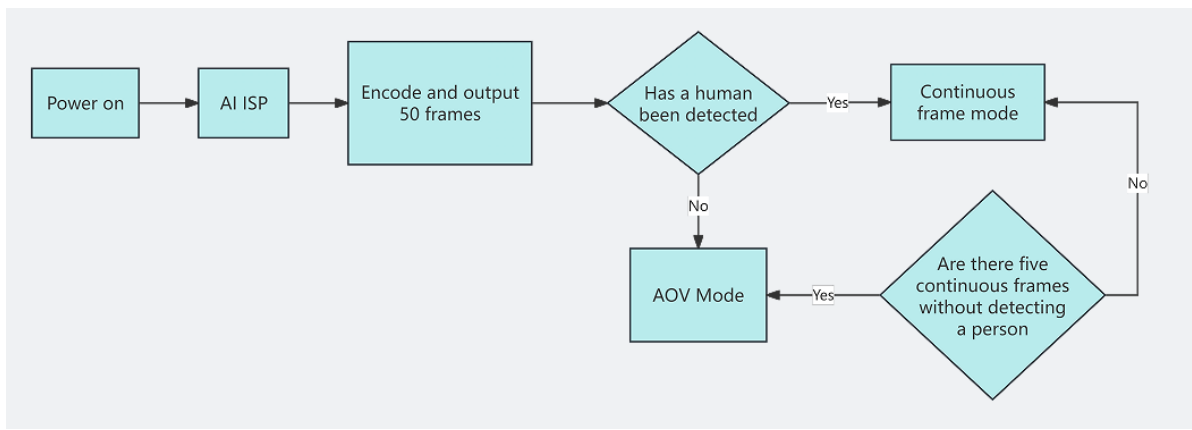
sample_aov_vi_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/

Option	Description	Default Value
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP function. If this option is used, enable the built-in ISP function. If no parameter is provided, the default value is used. Parameter is the path to the aiq file folder.	NULL
-e --encode	Set encoding method	h265vbr
-b --bitrate	Set encoding bitrate	4096
-r --wrap	Wrap switch, 0: off, 1: on	0
-f --fps	Set camera frame rate	25
--wrap_lines	Wrap line setting, 0: height/2, 1: height/4, 2: height/8	1
--suspend_time	System sleep duration, unit ms	1000
--aov_loop_count	Set the number of AOV output frames before exiting the program	-1
--boot_frame	Set the number of frames to enter AOV mode after running the program	60
--enable_multi_frame	Whether to enter continuous frame mode after human is detected, 1: enter continuous frame, 0: continue to sleep	1
--low_power	Low power mode, 0: off, 1: on	0
--camera_mirror_flip	Set camera rotation, 0: 0, 1: 90, 2: 180, 3: 270	0
--fast_ae	Enable fast exposure convergence, 0: off, 1: on	0

4.1.4 AIISP Testing

Description:

After AIISP processing, it is sent to NN for human recognition. If a human form is detected, it switches to continuous frame mode, and when it fails to recognize a human for 5 consecutive frames, it enters a sleep state.



Code Path:

project/app/aov_sample/sample_aov_aiisp_venc.c

Quick Usage:

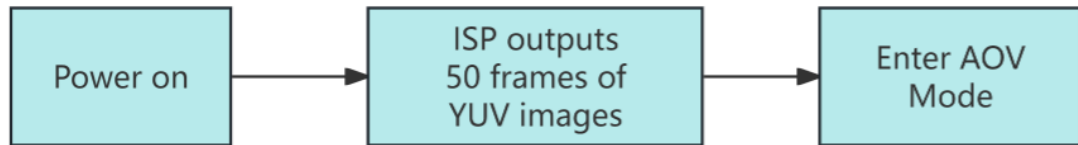
sample_aov_aiisp_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/ -l 0 -e h264cbr

Option	Description	Default Value
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP function. If this option is used, the built-in ISP function is enabled. If no parameter is provided, the default value is used. The parameter should be the path to the AIQ file directory.	NULL
-e --encode	Set the encoding method	h265vbr
-b --bitrate	Set the encoding bitrate	4096
-r --wrap	Wrap switch, 0: Off, 1: On	0
-f --fps	Set the camera frame rate	25
--wrap_lines	Wrap line setting, 0: height/2, 1: height/4, 2: height/8	1
--suspend_time	System sleep duration in milliseconds	1000
--aov_loop_count	Set the number of AOV output frames before exiting the program	-1
--boot_frame	Set the number of frames to enter AOV mode after running the program	60
--enable_multi_frame	Whether to enter continuous frame mode after human is detected, 0: Continue to sleep, 1: Enter continuous frame mode	1
--enable_aiisp	Whether to enable AIISP, 0: Off, 1: On	
--enable_aov	Whether to enable AOV mode, 0: Off, 1: On. If off, the stream can be previewed with RTSP at rtsp://xx.xx.xx.xx/live/0	1
--low_power	Low power mode, 0: Off, 1: On	1
--camera_mirror_flip	Set camera rotation, 0: 0, 1: 90, 2: 180, 3: 270	0
--fast_ae	Enable fast exposure convergence, 0: Off, 1: On	0

4.1.5 Dual Camera ISP Stream Capture Test

Description:

Capture a frame from each of the ISP0 and ISP1, and then enter sleep mode:



Code Path:

project/app/aov_sample/sample_aov_multi_vi.c

Quick Usage:

sample_aov_multi_vi -s 0 -w 2048 -h 1536 -s 1 -w 1920 -h 1080 -a /etc/iqfiles/ --aov_loop_count 10

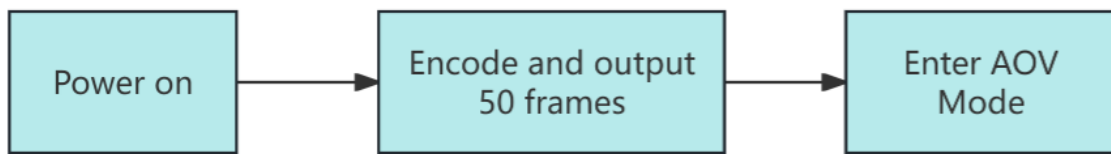
Option	Description	Default Value
-s	0 indicates the primary camera, 1 indicates the secondary camera, used to set the resolution for the corresponding camera	0
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP functionality, input this option to enable built-in ISP functionality, without parameters it uses default values, parameters are the path to the aiq file folder.	NULL
--suspend_time	System sleep duration, in milliseconds	1000
--aov_loop_count	Set how many frames to output before exiting the program	-1
--boot_frame	Set how many frames to run before entering AOV mode	60
--vi_frame_mode	Single frame or single frame multi-frame mode switch. 0: Single frame mode, 1: Single frame -> Multi-frame -> Single frame	0
--output_path	The output path for yuv, file name is vi_0.bin	NULL
--isp_mode	ISP working mode, 0: Readback, 1: Semi-Direct, 2: Full Direct. In readback mode, both ISPs require raw buffers, semi-direct only requires one raw buffer configuration, full direct does not require raw buffers. However, compared to readback mode, full direct and semi-direct have stricter requirements for exposure synchronization and vblank between dual camera sensors, not meeting the requirements will result in frame loss. Therefore, considering memory and effect comprehensively, semi-direct is used by default	1
--cif_buff_cnt	The number of raw buffers for the driver, effective for readback and semi-direct, if the ISP frame rate is insufficient, you can appropriately increase the number of this buffer	1
--enable_change_resolution	Enable switching between small and large resolutions. When entering AOV mode, switch the sensor and encoder output to a small image to reduce power consumption; switch to continuous frame mode to output a large image	0

Option	Description	Default Value
--enable_scale	When enabling the switching between small and large resolution functions, do not take the resolution of the encoder after switching, but instead scale up the large image sent to the backend encoding by ISP, which can reduce the complexity of the application, but relatively the power consumption in AOV mode will increase	0

4.1.6 Dual Camera Main Stream Encoding Test

Description:

Frames are taken from both ISP0 and ISP1 and send them for encoding. After all encoding is complete, enter sleep mode. If an SD card is inserted, the stream will be stored to `/mnt/sdcard/wakeup_frame_%d`, with a threshold of 10M. If there are more than 100 folders, manual deletion is required.



Code Path:

project/app/aov_sample/sample_aov_multi_vi_venc.c

Quick Usage:

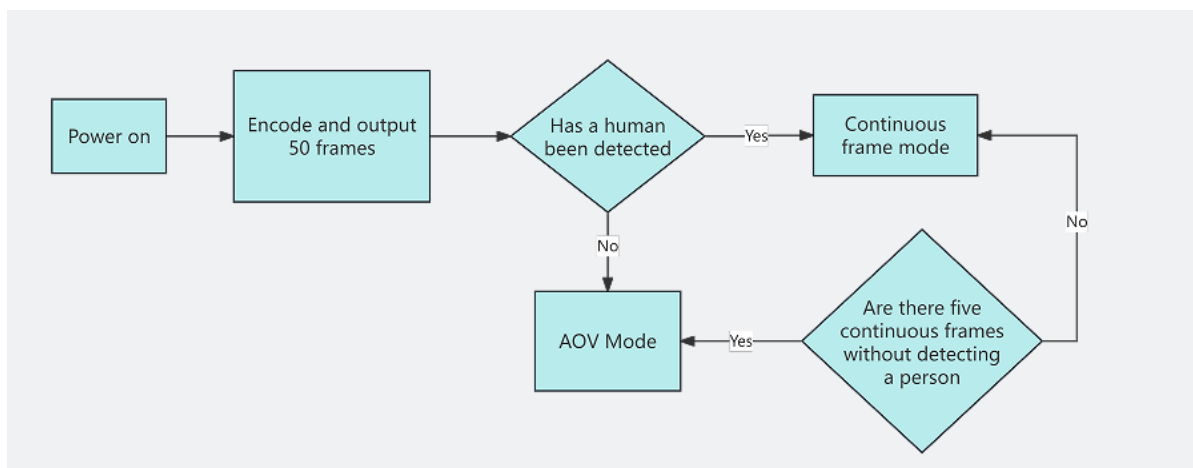
```
sample_aov_multi_vi_venc -s 0 -w 2048 -h 1536 -s 1 -w 1920 -h 1080 -a /etc/iqfiles/ --aov_loop_count 10
```

Option	Description	Default
-s	0 indicates the main camera, 1 indicates the sub-camera, used to set the resolution of the corresponding camera	0
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP functionality, input this option to enable built-in ISP functionality, without parameters, it uses default values, the parameter is the path to the aiq file folder.	NULL
-e --encode	Set encoding method	h264vbr
-b --bitrate	Set encoding bitrate	4096
-r --wrap	Wrap switch, 0: disable, 1: enable	0
--wrap_lines	Wrap line settings, 0: height/2, 1: height/4, 2: height/8	1
--suspend_time	System sleep time in ms	1000
--aov_loop_count	Set how many frames AOV outputs before exiting the program	-1
--boot_frame	Set how many frames to enter AOV mode after running the program	60
--enable_save_sdcard	Save stream to SD card	0

4.1.7 Dual-Camera Human Detection Test

Instructions:

Frames are sent from ISP0 and ISP1 to the encoder respectively. An asynchronous thread captures a small image and sends it to IVS for motion detection, followed by NN for human recognition. If a human is detected, it switches to continuous frame mode. After 5 seconds without detecting a human form, it enters sleep mode.



project/app/aov_sample/sample_aov_multi_vi_iva_venc.c

Quick Start:

sample_aov_multi_vi_iva_venc -s 0 -w 2048 -h 1536 -s 1 -w 1920 -h 1080 -a /etc/iqfiles/

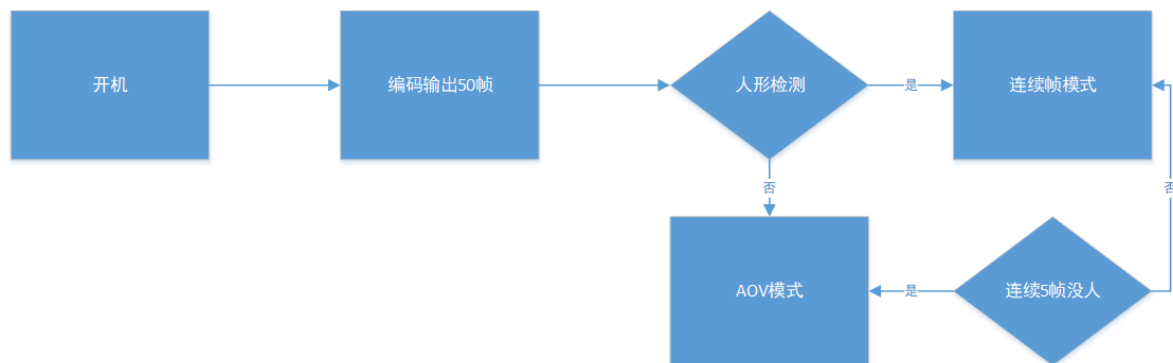
Option	Description	Default Value
-s	0 indicates the primary camera, 1 indicates the secondary camera, used to set the resolution of the corresponding camera	0
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP functionality, input this option to enable built-in ISP functionality, without parameters it uses default values, parameters are the path to the aiq file folder.	NULL
-e --encode	Set encoding method	h264vbr
-b --bitrate	Set encoding bitrate	4096
-r --wrap	Wrap switch, 0: off, 1: on	0
--wrap_lines	Wrap line setting, 0: height/2, 1: height/4, 2: height/8	1
--suspend_time	System sleep time in ms	1000
--aov_loop_count	Set how many frames AOV outputs before exiting the program	-1
--boot_frame	Set how many frames after running the program before entering AOV mode	60
--isp_mode	ISP working mode, 0: Readback, 1: Semi-Direct, 2: Full Direct. In readback mode, both ISPs require raw buffers, semi-direct only requires one raw buffer configuration, full direct does not require raw buffers. However, compared to readback mode, full direct and semi-direct have stricter requirements for exposure synchronization and vblank between dual camera sensors, not meeting the requirements will result in frame loss. Therefore, considering memory and effect comprehensively, semi-direct is used by default	1
--cif_buff_cnt	The number of raw buffers for the driver, effective for readback and semi-direct, if the ISP frame rate is insufficient, you can appropriately increase the number of this buffer	1
--vi_buff_cnt	The number of ISP output buffers, if the ISP frame rate is insufficient, you can appropriately increase the number of this buffer	1

Option	Description	Default Value
--enable_change_resolution	Enable switching between small and large resolutions. When entering AOV mode, switch the sensor and encoder output to a small image to reduce power consumption; when switching to continuous frame mode, output a large image	0
--enable_scale	When enabling the switch between small and large resolutions, do not take the encoder resolution after switching, but instead scale up the large image sent by ISP for encoding, which can reduce the complexity of the application, but relatively the power consumption in AOV mode will increase	0
--enable_save_sdcard	Save the bitstream to the SD card	0

4.1.8 Dual Camera Vertical Non-Blend Stitching Test

Description:

Capture one frame from ISP0 and one from ISP1, perform vertical stitching with ISP, and then send the stitched result to the encoder. An asynchronous thread captures a small image, sends it to IVS for motion detection, and then to NN for human recognition. If a human is recognized, it switches to continuous frame mode. If no human is recognized for 5 seconds continuously, it enters sleep mode.



project/app/aov_sample/sample_aov_multi_vi_nonblend_venc.c

Quick Start:

sample_aov_multi_vi_nonblend_venc -w 1920 -h 1080 -a /etc/iqfiles/

Option	Description	Default
-w	Resolution width	1920
-h	Resolution height	1080
-a --aiq	Enable built-in ISP functionality, input this option to enable built-in ISP functionality, without parameters it uses default values, parameters are the path to the aiq file folder.	NULL
-e --encode	Set encoding method	h264vbr
-b --bitrate	Set encoding bitrate	4096
-r --wrap	Wrap switch, 0: off, 1: on	0
--wrap_lines	Wrap line setting, 0: height/2, 1: height/4, 2: height/8	1
--suspend_time	System sleep duration, unit ms	1000
--aov_loop_count	Set how many frames AOV outputs before exiting the program	-1
--boot_frame	Set how many frames after running the program before entering AOV mode	60
--isp_mode	ISP working mode, 0: Readback, 1: Semi-Direct, 2: Full Direct. In readback mode, both ISPs require raw buffers, semi-direct only requires one raw buffer configuration, full direct does not require raw buffers. However, compared to readback mode, full direct and semi-direct have stricter requirements for exposure synchronization and vblank between dual camera sensors, not meeting the requirements will result in frame loss. Therefore, considering memory and effect comprehensively, semi-direct is used by default	1
--cif_buff_cnt	The number of raw buffers for the driver, effective for read-back and semi-straight-through, if the ISP frame rate is insufficient, you can appropriately increase the number of this buffer	1
--vi_buff_cnt	The number of ISP output buffers, if the ISP frame rate is insufficient, you can appropriately increase the number of this buffer	1
--enable_save_sdcard	Save the encoded stream to the SD card	0

4.2 Sleep&Wake-up Latency Troubleshooting

4.2.1 AOV Time Confirmation

Method 1: Connect the power_en pin to an oscilloscope, and observe the working duration;

Method 2: Using a stopwatch to confirm the average interval between camera frames.

4.2.2 Software Latency Troubleshooting

1. Disable the asynchronous serial port printing:

```
Disable CONFIG_RK_CONSOLE_THREAD option
```

2. Open PM-Log to check the overall driver sleep&wake-up latency. If a driver with excessive latency is found, refer to the SD card driver to prevent it from participating in the AOV sleep&wake-up process.

```
echo 0 > /sys/power/pm_async  
echo N > /sys/module/printk/parameters/console_suspend  
echo 1 > /sys/power/pm_print_times
```

4.3 Streaming Issues Troubleshooting

1. Confirm if the camera driver has been called until the stream is restored:

```
case RKMODULE_SET_QUICK_STREAM:  
    stream = *((u32 *)arg);  
    if (stream)  
        ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,  
                                SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);  
    else  
        ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,  
                                SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);
```

If logs have not been added, you can also read the sensor's streaming register to determine if it is working properly.

```
i2ctransfer -f -y 4 w2@0x30 0x01 0x00 r1
```

2. Check if there is stream output from the cif and isp nodes:

```
cat /proc/rkcif-mipi-lvds  
cat /proc/rkisp-vir0
```

3. In AOV mode, the entire media path cannot use the frame-dropping strategy.
4. If there is no stream output in the wrap mode, you can sequentially use the following three strategies:

Increase the Wrap Line

You can increase the wrap line to 1/2, or the entire buffer.

Increase the Encoder Frequency

```
sysdrv/source/kernel$ git diff arch/arm/boot/dts/rv1106.dtsi
diff --git a/arch/arm/boot/dts/rv1106.dtsi b/arch/arm/boot/dts/rv1106.dtsi
index 6180d18da..6830e75f1 100644
--- a/arch/arm/boot/dts/rv1106.dtsi
+++ b/arch/arm/boot/dts/rv1106.dtsi
@@ -76,7 +76,7 @@ cpu_pvtpll: cpu-pvtpll {

    rkvinc_pvtpll: pvtpll-0 {
        compatible = "fixed-clock";
-       clock-frequency = <410000000>;
+       clock-frequency = <500000000>;
        clock-output-names = "clk_pvtpll_0";
        #clock-cells = <0>;
    };
@@ -1272,9 +1272,9 @@ rkvinc: rkvinc@ffa50000 {
    interrupt-names = "irq_rkvinc";
    clocks = <&cru ACLK_VEPU>, <&cru HCLK_VEPU>, <&cru CLK_CORE_VEPU>;
    clock-names = "aclk_vcodec", "hclk_vcodec", "clk_core";
-   rockchip,normal-rates = <300000000>, <0>, <410000000>;
+   rockchip,normal-rates = <300000000>, <0>, <500000000>;
    assigned-clocks = <&cru ACLK_VEPU>, <&cru CLK_CORE_VEPU>;
-   assigned-clock-rates = <300000000>, <400000000>;
+   assigned-clock-rates = <300000000>, <500000000>;
    resets = <&cru SRST_A_VEPU>, <&cru SRST_H_VEPU>, <&cru SRST_CORE_VEPU>;
    reset-names = "video_a", "video_h", "video_core";
    rockchip,srv = <&mpp_srv>;
```

After modifying the dts, you need to enter this command for it to take effect.

```
echo clk_core_vepu 500000000 > /proc/clock/rate
```

Decrease the ISP Frequency

```
diff --git a/drivers/media/platform/rockchip/isp/hw.c b/drivers/media/platform/rockchip/isp/hw.c
index 6457772f7b89..aaa1647b309f 100644
--- a/drivers/media/platform/rockchip/isp/hw.c
+++ b/drivers/media/platform/rockchip/isp/hw.c
@@ -683,10 +683,10 @@ static const struct isp_clk_info rv1106_isp_clk_rate[] = {
    .clk_rate = 200,
    .refer_data = 2688,
}, {
-   .clk_rate = 350,
+   .clk_rate = 200,
    .refer_data = 3072,
}, {
-   .clk_rate = 440,
+   .clk_rate = 200,
    .refer_data = 3840,
}
};
```

5. Typical Power Consumption

5.1 Power Consumption During Basic Sleep

5.1.1 RV1106

When the controller is in sleep mode without peripherals, the overall power consumption should be around 5mW. If the test data does not match, it is necessary to check for any leakage to the controller. Please refer to the section on PM-GPIO configuration for details as follows:

1. Test Commands

```
io -4 0xff300048 320000000
echo mem > /sys/power/state
```

2. Power Consumption Data

Circuit	Current
3V3_STB	Approximately 0.05mA
0V9_STB	Approximately 0.15mA (RTC) / Approximately 0.41mA (PVTM)
DDR_1V36	Approximately 1.8mA

5.1.2 RV1106B

The power consumption of the entire device without peripherals in sleep mode should be around 4.1mW. If the test data does not match, it should be checked whether there is any leakage to the SoC side. Please refer to the section on PM-GPIO configuration for details as follows:

1. Test command

```
io -4 0x20514094 320000000
echo mem > /sys/power/state
echo mem > /sys/power/state #need to enter twice to ensure that the device is in sleep mode
```

2. Power consumption

Circuit	Current
3V3_STB	About 0.1mA
DDR_1V36	About 1.8mA

5.2 RV1106 SC200AI data

5.2.1 Encoding a Frame + Human Detection

1. Test Command

```
sample_aov_vi_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/ --enable_multi_frame 0 --suspend_time 1000 --low_power 1
```

- `--enable_multi_frame 0` : Indicates that it will not enter continuous frame mode when detecting a human; `1` : Indicates that it will enter continuous frame mode when detecting a human.
- `--suspend_time 1000` , specifies the sleep duration in milliseconds.
- `--low_power` , enables low power mode.

2. Actual Power Consumption Data

Sleep Duration	Work Duration	Total Power Consumption
1000ms	75ms	39mW
500ms	75ms	66mW
330ms	75ms	92mW

5.2.2 AIISP + Encoding a Frame + Human Detection

1. Test Command

```
sample_aov_aiisp_iva_venc -w 1920 -h 1080 -a /etc/iqfiles/ --suspend_time 1000 --enable_multi_frame 0
```

- `--enable_multi_frame 0` : Indicates that it will not enter continuous frame mode when detecting a human; `1` : Indicates that it will enter continuous frame mode when detecting a human.
- `--suspend_time 1000` , specifies the sleep duration in milliseconds.

2. Actual Power Consumption Data (When AIISP is Enabled and Operational)

Sleep Duration	Working Duration	Overall Power Consumption
1000ms	75ms	76mW
500ms	75ms	129mW
330ms	75ms	170mW

5.3 RV1106B SC450AI Performance

5.3.1 Encode one frame + human detection

1. Test command

```
sample_aov_vi_iva_venc -w 2688 -h 1520 -a /etc/iqfiles/ --enable_save_mp4 1 --enable_bypass 1 -r 1 --aov_loop_count -1 --camera_mirror_flip 0 --suspend_time 1000
```

- --suspend_time 1000, specifies the sleep time, in ms

2. Power consumption test data

Frame interval	Working time	Power consumption of the whole device
1000ms	66ms	34mW
2000ms	66ms	19.3mW
3000ms	66ms	14.1mW

5.4 RV1106B Dual-SC200AI Performance

5.4.1 Non-blending stitching + encode one frame + human detection

1. Test command

```
sample_aov_multi_vi_nonblend_stitch_venc --suspend_time 1000 &
```

- --suspend_time 1000, specifies the sleep time, in ms

2. Power consumption test data

Frame interval	Working time	Power consumption of the whole device
1000ms	98ms	55.7mW
2000ms	98ms	30.8mW
3000ms	98ms	22.6mW

6. Frequently Asked Questions

6.1 How to Add Sleep Time to Application Layer Timing

```
struct timespec t;  
clock_gettime(CLOCK_BOOTTIME, &t);
```

6.2 How to Obtain the Wakeup Source

1. If registered as a button event, you can refer to the following interface:

```
bool SAMPLE_COMM_AOV_GetGpioIrqStat();
```

2. You can check through the following node:

```
cat sys/kernel/wakeup_reasons/last_resume_reason
```

6.3 Why the SD Card is Not Recognized After AOV SDK Boot

The SDK defaults to unloading the SD card driver, which can be reloaded using the following command:

```
# rv1106
echo "ffaa0000.mmc" > /sys/bus/platform/drivers/dwmmc_rockchip/bind
# rv1106b
echo "20d20000.mmc" > /sys/bus/platform/drivers/dwmmc_rockchip/unbind
```

6.4 How to Resolve Data Transmission Issues During AOV Process with SD Card or 4G When Data is Temporarily Unavailable

The SDK provides the following interface, which can be called manually by the application to output a single frame in single-frame mode, with each call resulting in one frame being output:

```
RK_MPI_VI_DevEnableSingleFrame
```

6.5 How to Control Infrared Lights with Lower Power Consumption

During the AOV process, the infrared light does not need to be on continuously. The switch of the infrared light can be controlled through the following interface:

```
RK_MPI_VI_DevEnableLight
```

Note: The interface enables the infrared light, and this method cannot be used to control the infrared light through export or other external method. For camera driver development, refer to the **Fill light Control** section.